CPCS361 SECTION CS2 GROUP 3

CPU Scheduling Simulator

Dynamic & Static Round Robin

Sultan Yasir Alasami
ID: 2339663

Abdulaziz Sattam Alobaili
ID: 2446346

Faisal Hussein Alghamdi
ID: 2337220

College: FCIT | Supervised by: Dr. ASSAD

Introduction

This project focuses on the core challenge of operating systems: efficiently managing access to the Central Processing Unit (CPU).

CPU scheduling is a critical OS component determining how processes access the processor. Efficient scheduling optimizes performance, minimizes wait times, and ensures fairness.

Project Goal

To develop a simulator modeling **Dynamic Round Robin (DRR)** behavior and compare it against Static Round Robin under various system configurations, demonstrating the benefits of adaptive scheduling.

Project Objectives

- **Implement DRR Algorithm:** Create a functional simulator supporting dynamic time quantum calculation.
- **Simulate Resource Constraints:** Model realistic process arrivals, memory allocation, and device allocation.
- **Parameter Analysis:** Observe how memory size and burst times affect scheduling.
- **Performance Comparison:** Quantitatively analyze Turnaround Time and Waiting Time against Static Round Robin.

System Architecture

Process Manager

Manages PCB data, process state (Ready, Hold), and prioritizes resource admission.

Queues

Ready Queue: FIFO for active jobs.
Hold Queues: Jobs waiting for memory/devices (Priority & FIFO).

Scheduler

Executes DRR logic: dynamically calculates Time Quantum before dispatching the next process.

Controller

Parses input, manages the global clock, and synchronizes internal and external events.

Static vs. Dynamic Round Robin

Static Round Robin (SRR)

Uses a fixed time quantum regardless of the current system workload or the nature of processes.

```
Quantum (Q) = Constant Value
```

Drawback: Poor efficiency in mixed environments; high context switching for small Q or long waiting times for large Q.

Dynamic Round Robin (DRR)

Adapts the time quantum based on the average remaining burst time of processes in the Ready Queue.

```
Quantum (Q) = Sum Of Remaining Time /
                Process Count
```

Benefit: Improves throughput and reduces waiting time by granting proportional slices.

Process Flow Lifecycle

1	2	3	4	5
Arrival	**Hold Queue**	**Ready Queue**	**Execution**	**Finish**
Job enters the system, requesting memory and I/O devices.	If resources are unavailable, the job waits in HQ1 (Priority) or HQ2 (FIFO).	Resources allocated. Job waits for its turn to be selected by the CPU scheduler.	CPU runs the job for the dynamic Time Quantum or until completion.	Job completes. Resources are released, potentially unblocking waiting jobs.

Code Implementation (High-Level)

SimulationController

Acts as the **main event loop**, advancing the global clock and deciding between internal (completion/preemption) or external (job arrival) events.

PrManager

Contains the core OS logic: managing the Ready/Hold queues and executing the `admitFromHolds` logic when memory is freed.

DRoundRobinScheduler

Extends the base scheduler to implement the dynamic quantum formula, ensuring the time slice adapts to the current workload of the Ready Queue.

OtherKerServices

Manages the total system resources (Memory and Devices) and handles the allocation and deallocation process for arriving and finishing jobs.

Performance Results (M=200)

With sufficient memory (M=200), DRR significantly outperforms SRR by adapting to the workload mix.

DRR was **12.4% faster** in Turnaround Time and **14.2% faster** in Waiting Time.

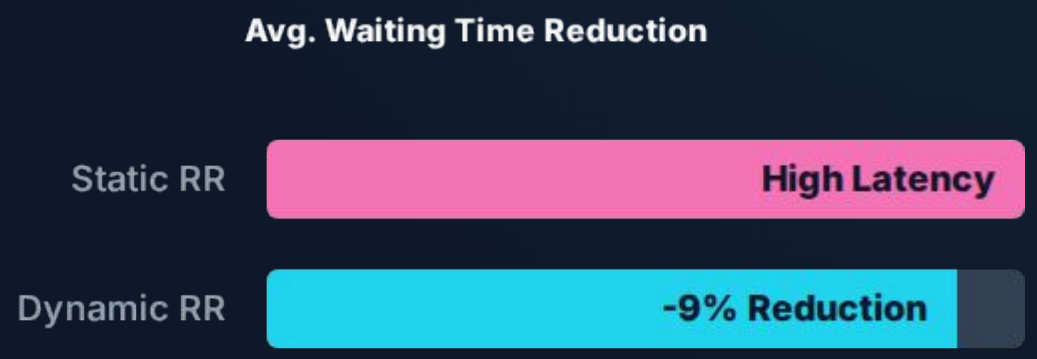
Special Case Analysis: The "Convoy Effect"

Scenario

Simulating a heavy workload mix: 10 long jobs (100ms) and 40 short jobs (2ms). This often causes short, interactive tasks to suffer high latency.

DRR Outcome

DRR detected the low average remaining time in the queue and significantly shrank the quantum, effectively mitigating the convoy effect.



DRR successfully prevented process starvation for short jobs.

Project Concluion

"Dynamic Round Robin excels in mixed workloads by adapting to the queue state, effectively reducing waiting times for short processes compared to Static RR."

Key Takeaway

Improved Responsiveness

Enhanced Fairness

Future Work

- **Advanced Scheduling:** Integrate multi-level feedback queue (MLFQ) using DRR as the base scheduling algorithm.
- **Resource Modeling:** Implement a more complex memory management unit (MMU) simulation with fragmentation and swapping.
- **Visualization:** Develop a Gantt chart module to visually track process execution and context switches in real-time.
- **Network Simulation:** Extend the simulator to handle distributed processes and remote procedure calls (RPC).

Q & A

Thank you for your attention.

Team 3 | CPCS361